Efficient Image Retrieval Using Hierarchical K-Means Clustering

TEAM PROJECT

Subject : MACHINE LEARNING

Members:

- Ronit Dutta (1010683)
- Nan Li (1009421)
- Mohamed Mukhtar (1009406)
- Afifah Mesha Putri (1010690)
- Timothy Wee (1005545)

sensors

MDPI

Article

Efficient Image Retrieval Using Hierarchical K-Means Clustering

Dayoung Park and Youngbae Hwang *

Department of Control and Robot Engineering, Chungbuk National University, Cheongju 28644, Republic of Korea; wnidy100@cbnu.ac.kr
* Correspondence: ybhwang@cbnu.ac.kr; Tel.: +82-43-261-3641

Abstract: The objective of content-based image retrieval (CBIR) is to locate samples from a database that are akin to a query, relying on the content embedded within the images. A contemporary strategy involves calculating the similarity between compact vectors by encoding both the query and the database images as global descriptors. In this work, we propose an image retrieval method by using hierarchical K-means clustering to efficiently organize the image descriptors within the database, which aims to optimize the subsequent retrieval process. Then, we compute the similarity between the descriptor set within the leaf nodes and the query descriptor to rank them accordingly. Three tree search algorithms are presented to enable a trade-off between search accuracy and speed that allows for substantial gains at the expense of a slightly reduced retrieval accuracy. Our proposed method demonstrates enhancement in image retrieval speed when applied to the CLIP-based model, UNICOM, designed for category-level retrieval, as well as the CNN-based R-GeM model, tailored for particular object retrieval by validating its effectiveness across various domains and backbones. We achieve an 18-times speed improvement while preserving over 99% accuracy when applied to the In-Shop dataset, the largest dataset in the experiments.

Keywords: image retrieval; CBIR; efficiency; hierarchical clustering; tree search

check for updates

Citation: Park, D.; Hwang, Y. Efficient Image Retrieval Using Hierarchical K-Means Clustering. *Sensors* **2024**, *24*, 2401. https://doi.org/10.3390/s24082401

Academic Editors: Stefano Berretti, Jean-Baptiste Thomas, Baptiste Magnier and Khizar Hayat

Received: 13 January 2024
Revised: 2 April 2024
Accepted: 8 April 2024
Published: 9 April 2024

1. Introduction

The modern field of CBIR is categorized into particular object retrieval and category-level retrieval [1–5]. In recent CBIR systems, images are typically represented as global feature vectors, and inter-vector distance metrics such as cosine [3] and Euclidean [6] distances are used to determine the similarity between a query and the database. Extensive studies on CBIR have been conducted on backbone networks utilizing various deep learning models, including convolutional neural networks (CNN) [2,3,7–12], vision transformers (ViT) [5,13–15], and graph convolutional networks (GCN) [16,17], as encoders for image vector representations.

One of the significant challenges in the image retrieval task is the retrieval time and computational cost. If the similarity between database and query descriptors is calculated without the use of any indexing technology, the time and computational resources required can become prohibitive, especially when dealing with large-scale datasets such as those found on the Internet, where the number of stored image samples is increasing exponentially, including video materials consisting of frame-by-frame images. Therefore, it is essential to employ an appropriate database classification or an indexing scheme for the practical application of image retrieval systems in real-world scenarios. Particularly in applications that demand real-time image retrieval, such as visual localization, where camera sensors are used to estimate location, the reduction of retrieval speed becomes a critical priority. Research focused on enhancing image retrieval accuracy is a dynamic and diverse field [4,18]. In contrast, studies aimed at improving retrieval speed show a less dynamic trend, with recent research primarily leaning towards hashing-based techniques [4,19–21].

Recognizing the need for progress in accelerating image retrieval, we explored the

OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

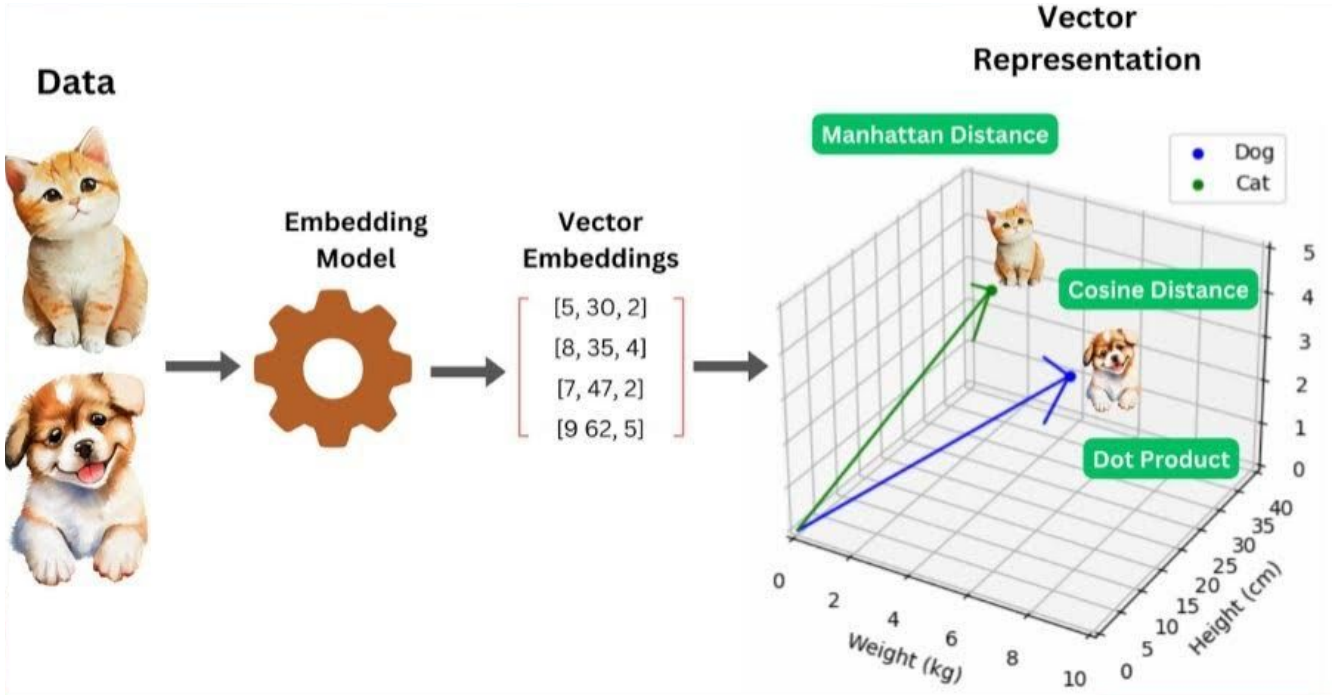
OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

Introduction

Content-based Image Retrieval (CBIR): allows users to retrieve images from a large scale dataset, by describing the content of the image.

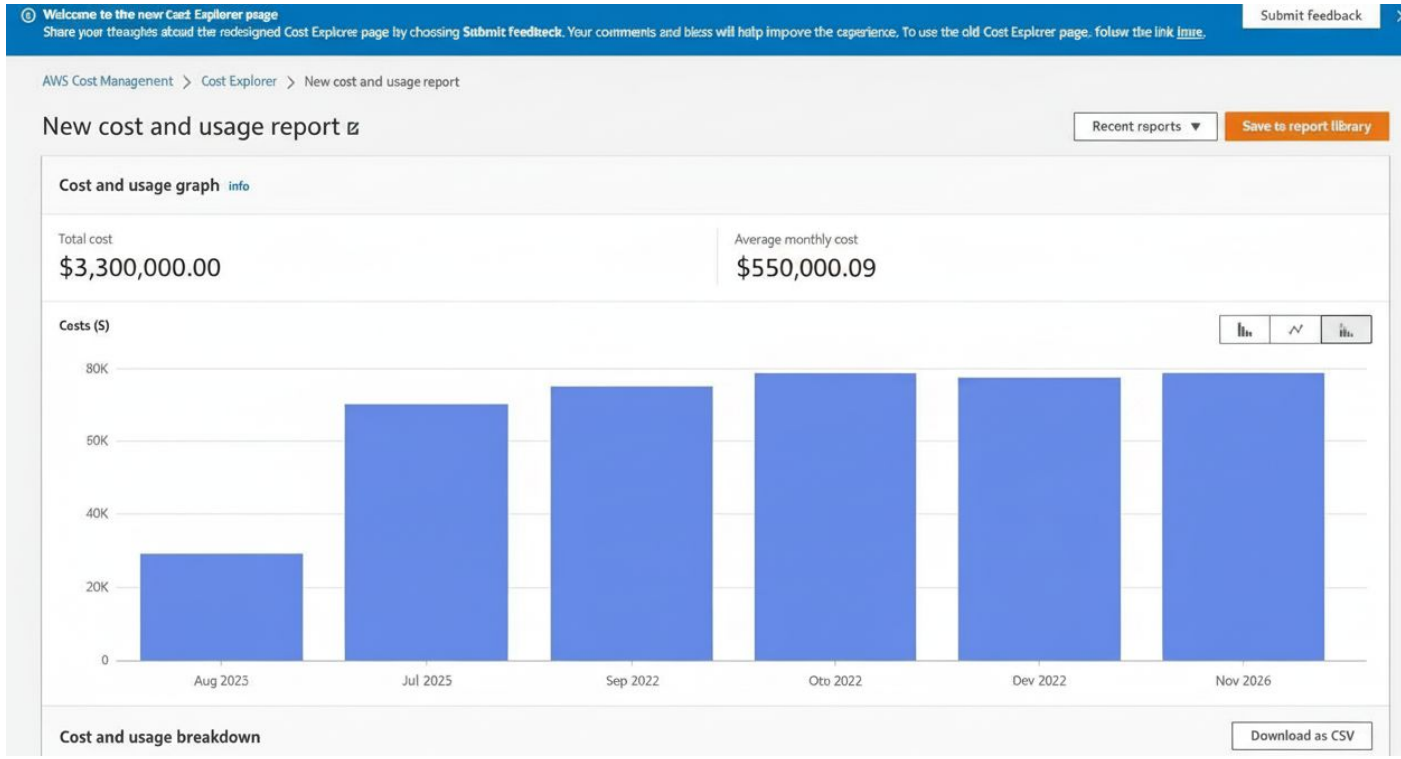
General approach: Modern CBIR strategies represent images as compact vectors, then utilise image-query similarity search.



Introduction

Challenge: As image datasets grow extremely large, most CBIR strategies becomes slow and computationally expensive.

Research question: how can we perform CBIR more quickly, with a lower computational cost?

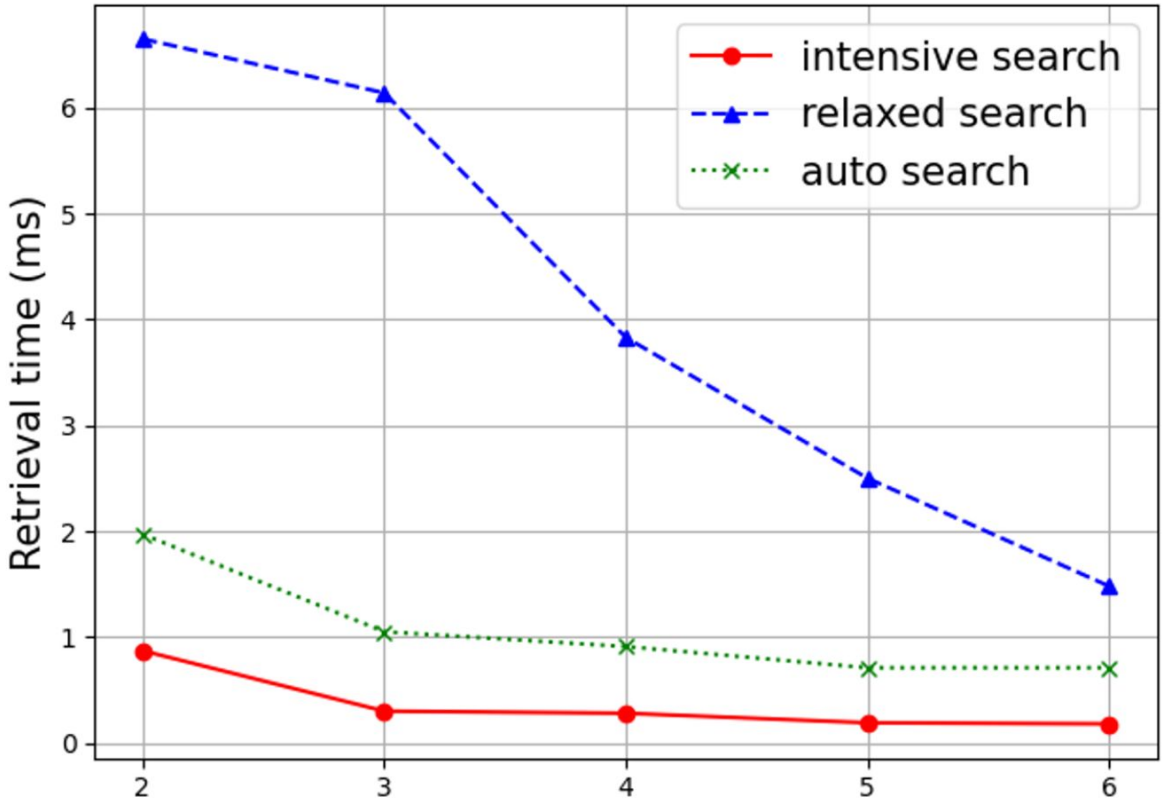


Core Motivations

1. **Computational Cost:** Computing similarity between query and all database images is prohibitively expensive
2. **Scale:** Internet-scale datasets with exponentially growing images and videos
3. **Real-time Requirement:** Fields like visual localisation in robotics need fast retrieval
4. **Research Gap:** While much is done to improve accuracy, retrieval speed improvements lag behind. Speed is needed for real-world applications

Speed-Accuracy Tradeoff

1. Three search algorithms enable flexible optimisation:
2. **Intensive Search:** Selects 1 node per level → fastest, potential accuracy loss
3. **Relaxed Search:** Selects multiple nodes → best accuracy preservation, slower
4. **Auto Search:** Automatic threshold detection → middle ground, no need for manual hyperparameter tuning
5. **Generality**: Works with CLIP-based models (UNICOM) and CNN-based models (R-GeM)



Data Selection

For categorical-level retrieval, three datasets were chosen:

Caltech-UCSD Birds (CUB) - 11,788 images of 200 bird species

The Stanford Cars dataset (CARS) - 8,131 images categorized into 98 car types.

Inshop - 12612 clothing item images, 3985 classes

These two datasets are **standard test datasets,** allowing authors to **benchmark accuracy and speed** of their approach against more traditional approaches.

***Caltech-UCSD Birds (CUB)** - 11,788 images of 200 bird species*

Data Selection

For Particular Object Retrieval, four datasets were chosen:

Oxford - 5063 images, 11 classes

Paris - 6392 images, 11 classes

The Oxford and Paris datasets are well-known benchmarks used in particular object image retrieval tasks.

Both datasets contain images categorised into landmark or object classes, and include query images for evaluation.

***Oxford Dataset** - 5063 images, 11 classes*

Data Selection

Two newer datasets were also used:

ROxford - Revisited version of Oxford with 4993 images, 13 categories, 70 queries.

RParis - Revisited version of Paris with 6322 images, 12 classes, 70 queries.

Revisited datasets (ROxford and RParis) stratify queries into easy, medium, and hard, reflecting increasing difficulty in retrieval due to factors like occlusion or distortion.

RParis *- 6322 images, 12 classes*

Dataset & Descriptor Statistics

- Consider a generic dataset index i (e.g. CUB, CARS, In-Shop, Oxford, ...):

$$\mathcal{D}_i = \{(d_j, y_j)\}_{j=1}^{N_i}$$

  - d_j: image
  - y_j: label (class for category-level, instance/object ID for object-level)
  - $N_i = |\mathcal{D}_i|$: dataset size
- Let $v_j \in \mathbb{R}^{\Delta}$ be the **global descriptor** of image d_j.
- Empirical descriptor mean and covariance:

$$\mu_i = \frac{1}{N_i} \sum_{j=1}^{N_i} v_j, \qquad \Sigma_i = \frac{1}{N_i} \sum_{j=1}^{N_i} (v_j - \mu_i)(v_j - \mu_i)^\top$$

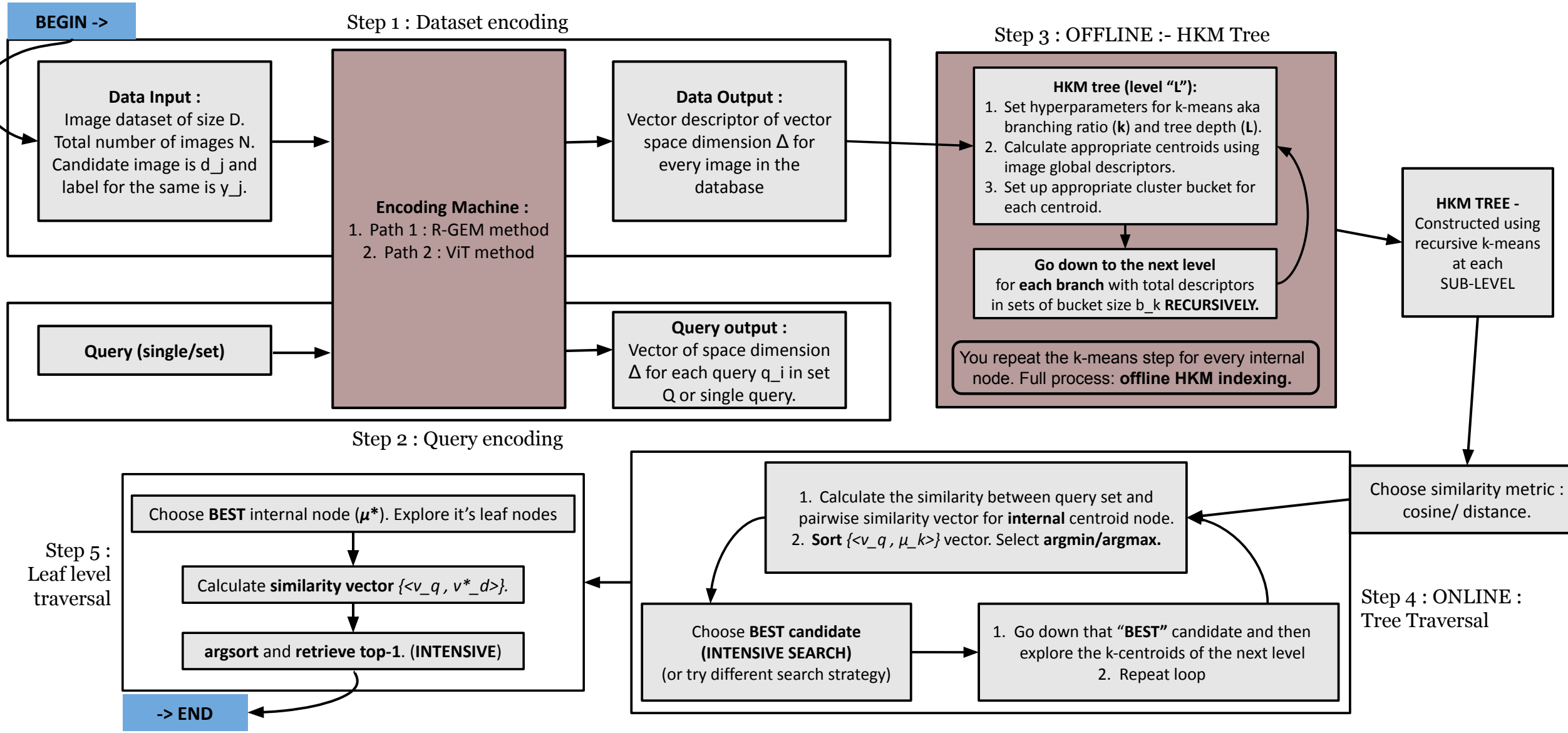
- Per-dimension standard deviation:

$$\sigma_i = \sqrt{\mathrm{diag}(\Sigma_i)}.$$

OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

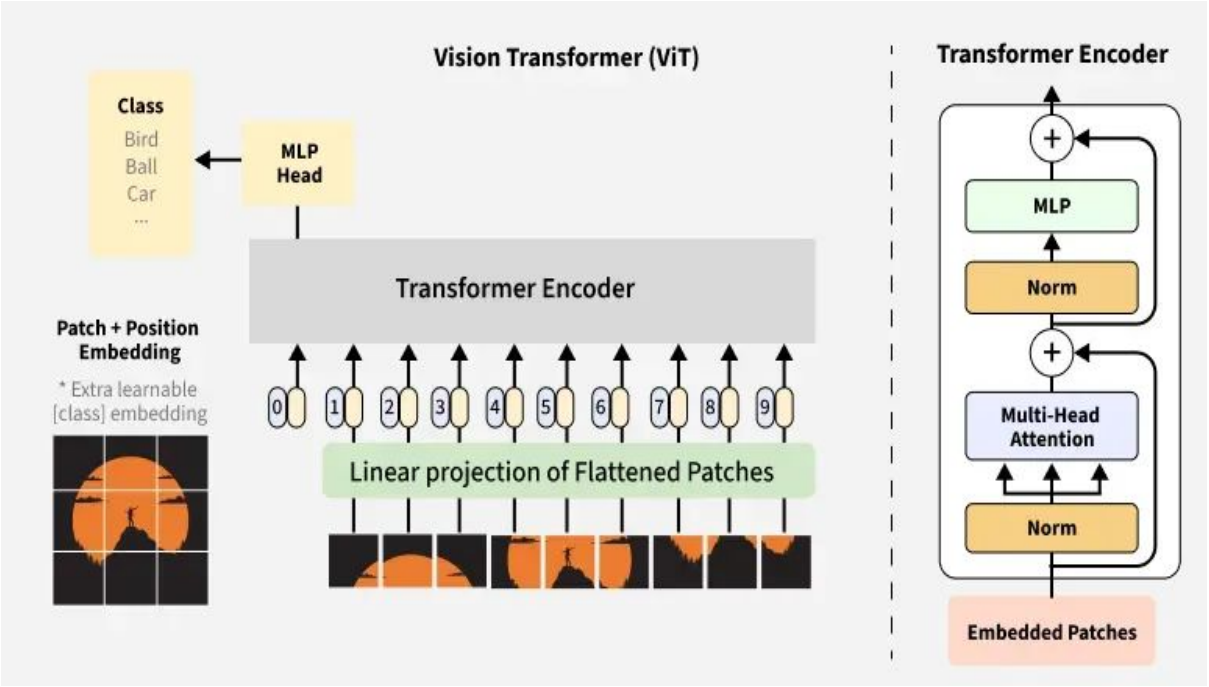
Implementation Mechanics review — STEP 1



Retrieval strategies

	Category Level Retrieval	**Particular Object Level Retrieval**
Goal / Intuition	Retrieve images with the same semantic category. (semantic / content similarity)	Retrieve the same specific instance/object (e.g. exact landmark). (instance-level, fine-grained appearance matching.)
Dataset	CUB, CARS, In-Shop.	Oxford, Paris, ROxford, RParis.
Backbone	UNICOM (CLIP/ViT (*Vision Transformer*)-based).	R-GeM (CNN + GeM pooling).
Metric	Recall@1.	mAP (mean Average Precision)

STEP 1 : Encoding - ViT based (category wise retrieval)



Given image d_j, patch embedding:

$$x_{j,1}, \ldots, x_{j,P} = \mathrm{PatchEmbed}(d_j), \quad x_{j,p} \in \mathbb{R}^D.$$

Add learnable CLS token and form sequence:

$$x_{j,\mathrm{cls}}^{(0)} \in \mathbb{R}^D, \quad X_j^{(0)} = [x_{j,\mathrm{cls}}^{(0)}, x_{j,1}, \ldots, x_{j,P}].$$

Pass through L Transformer layers:

$$X_j^{(L)} = T^{(L)} \circ \cdots \circ T^{(1)}(X_j^{(0)}).$$

Extract final CLS token:

$$z_j = x_{j,\mathrm{cls}}^{(L)} \in \mathbb{R}^D.$$

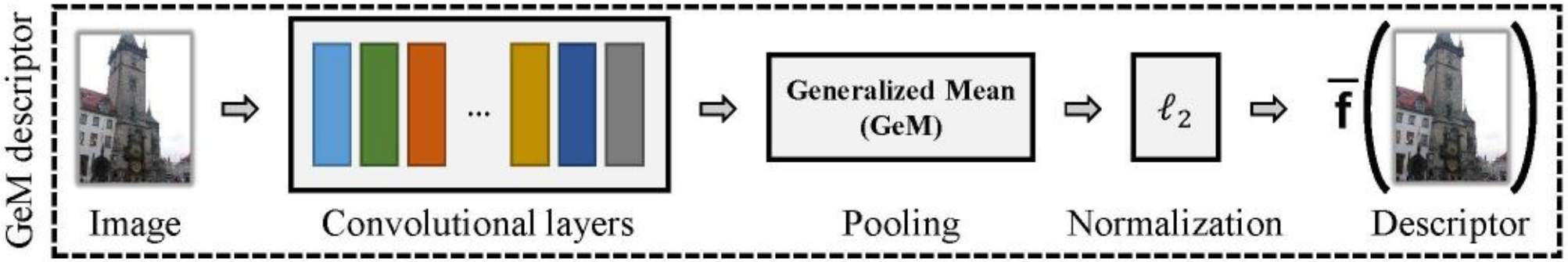
Projection + normalization:

$$v_j = \frac{W z_j + b}{\|W z_j + b\|_2} \in \mathbb{R}^\Delta.$$

Descriptor map (ViT/CLIP path):

$$v_j = \phi_{\mathrm{ViT}}(d_j).$$

STEP 1 : Encoding - CNN based (object retrieval)



1. Given image d_j, CNN backbone:

$$F_j = f_{\text{CNN}}(d_j) \in \mathbb{R}^{C \times H \times W}$$

- C: channels, $H \times W$: spatial resolution.

2. GeM pooling over spatial positions:

$$g_{j,c} = \left(\frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} F_j(c,h,w)^p \right)^{1/p}, \quad c = 1, \ldots, C$$

- Collect into:

$$g_j \in \mathbb{R}^C.$$

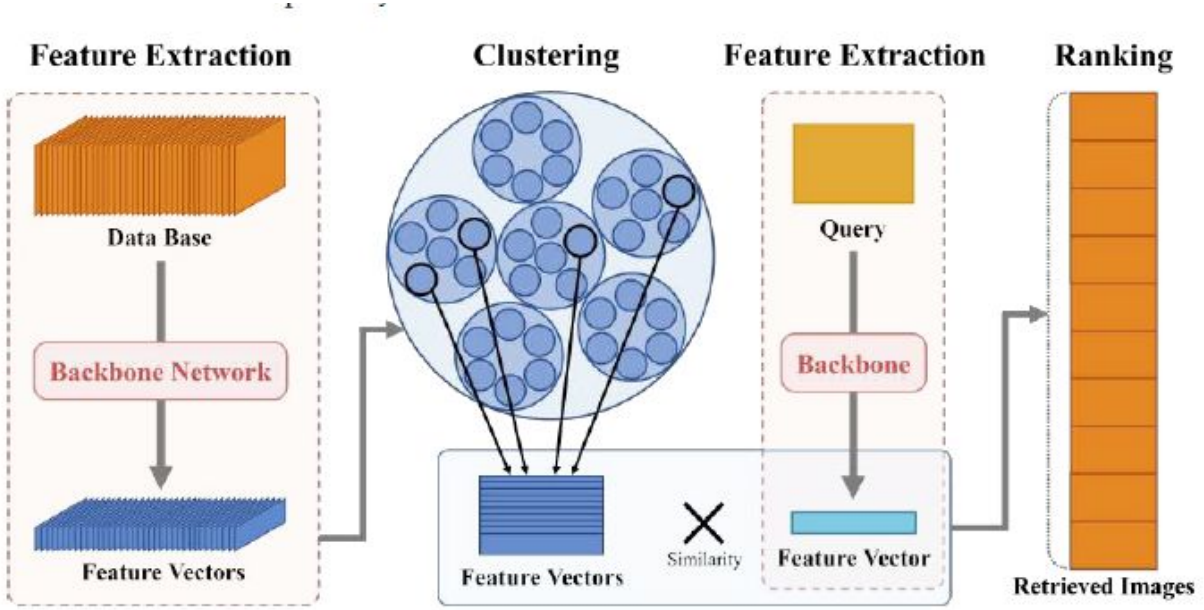
3. Projection + L2 normalization:

$$v_j = \frac{W g_j + b}{\|W g_j + b\|_2} \in \mathbb{R}^{\Delta}.$$

4. Descriptor map (CNN path):

$$v_j = \phi_{\text{CNN}}(d_j).$$

STEP 2 : Query Encoding



Let the **query image** be d_Q.

Use the **same encoder** as for database images:

Category-level (ViT/CLIP):

$$v_Q = \phi_{\text{ViT}}(d_Q) \in \mathbb{R}^{\Delta}.$$

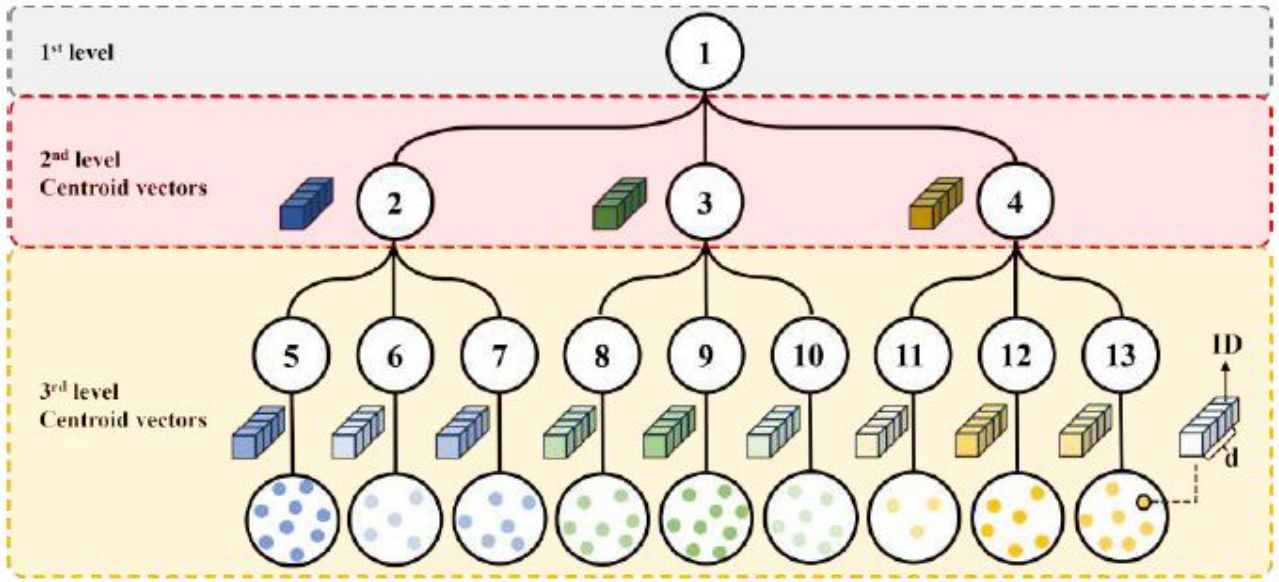
Particular-object (CNN):

$$v_Q^{\bullet} = \phi_{\text{CNN}}(d_Q) \in \mathbb{R}^{\Delta}.$$

Key requirement:

- Query descriptors and database descriptors are in the **same feature space** → comparable via cosine or Euclidean distance.

STEP 3 : Offline - HKM tree building



- After encoding all images:

$$\mathcal{V} = \{v_1, \ldots, v_{N_i}\}, \quad v_j \in \mathbb{R}^{\Delta}.$$

- Hyperparameters for Hierarchical K-Means (HKM):
  - Branching factor: k
  - Tree depth: L (paper uses l).
- Each node n in the tree stores:
  - Assigned descriptors: $S_n \subseteq \mathcal{V}$.
  - If **internal**: centroids $\{\mu_{n,1}, \ldots, \mu_{n,k}\}$.
  - If **leaf**: only its descriptor set S_n.

- For node n at level ℓ, run **k-means** on S_n:

$$\min_{\{C_{n,r}\}_{r=1}^{k}} \sum_{r=1}^{k} \sum_{v \in C_{n,r}} \|v - \mu_{n,r}\|_2^2$$

where

$$\mu_{n,r} = \frac{1}{|C_{n,r}|} \sum_{v \in C_{n,r}} v, \quad r = 1, \ldots, k.$$

- Define child nodes:
  - Child n_r at level $\ell + 1$ with:

$$S_{n_r} = C_{n,r},$$

$$b_{n,r}^{(\ell+1)} = |S_{n_r}|.$$

STEP 4 : ONLINE STAGE

OFFLINE STAGE

- At level ℓ, labeling nodes as $\psi_k^{(\ell)}$, bucket sizes:

$$b_k^{(\ell)} = |S_{\psi_k^{(\ell)}}|,$$

with:

$$\sum_{\psi_k^{(\ell)} \text{ at level } \ell} b_k^{(\ell)} = N_i.$$

- **Recursion until depth L:**
  - Repeat clustering for all internal nodes at $\ell = 0, \ldots, L-1$.
  - At $\ell = L$: nodes become **leaf nodes**, no further clustering.

ONLINE STAGE

- Given query descriptor v_Q, we perform **tree-based search.**
- Similarity / distance options:
- **Cosine similarity:**

$$\text{sim}(a, b) = \frac{a^\top b}{\|a\|_2 \|b\|_2}.$$

- **Euclidean distance:**

$$\text{dist}(a, b) = \|a - b\|_2.$$

- Retrieval objective:
  - Either **maximize** similarity $\text{sim}(v_Q, v_j)$,
  - Or **minimize** distance $\text{dist}(v_Q, v_j)$.

STEP 4 : ONLINE STAGE

- Start at root node, set $n \leftarrow \text{root}$.
- At internal node n, with child centroids:

$$\{\mu_{n,1}, \ldots, \mu_{n,k}\},$$

compute distances:

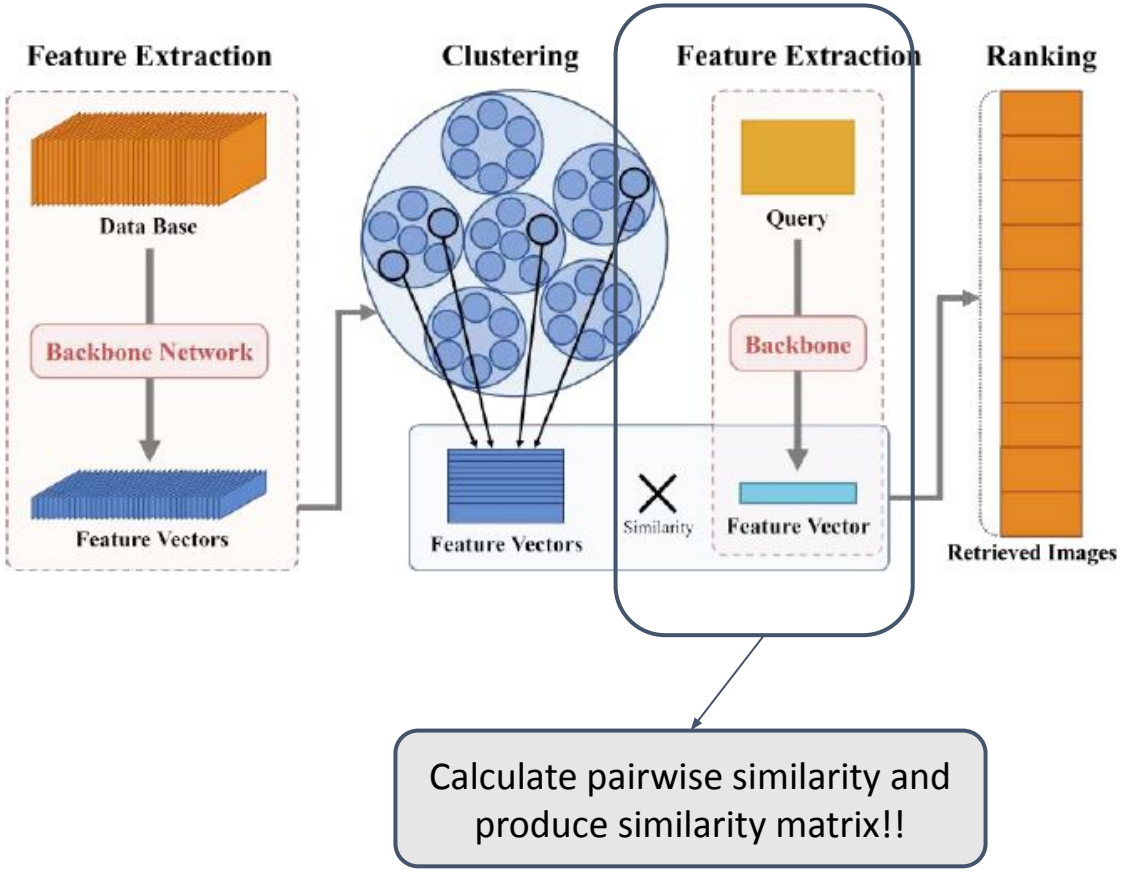
$$d_r = \text{dist}(v_Q, \mu_{n,r}), \quad r = 1, \ldots, k.$$

- Select best child index:

$$r^{\backslash *} = \arg\min_r d_r.$$

- Move to child:

$$n \leftarrow n_{r^{\backslash *}}.$$



STEP 5 : Final Retrieval

Now we select the best child node and then traverse the "**leaf**" image descriptors inside the bucket of the "**best**" internal node.

- Stack scores into a similarity vector:

$$\mathbf{s} = [s_1, \ldots, s_M].$$

- Define ranking permutation by sorting:

$$\pi = \text{argsort}(\mathbf{s}) \quad \text{(descending if using similarity)}.$$

- **Top-1 retrieved index:**

$$t^{\backslash *} = \pi_1.$$

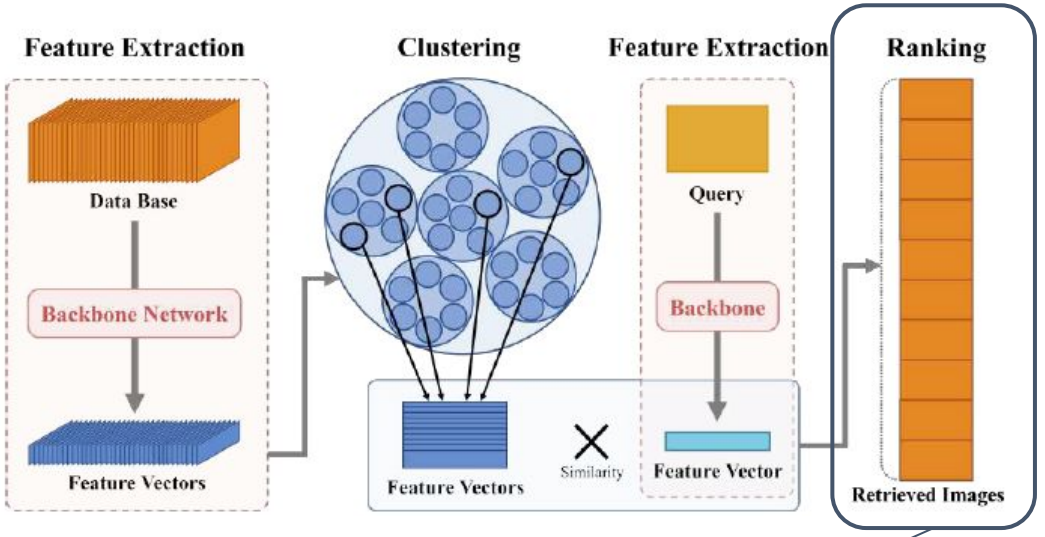
- **Best match image:**

$$d^{\backslash *} = d_{j_{t^{\backslash *}}}.$$

- For **top-K** retrieval:

$$(d_{j_{\pi_1}}, \ldots, d_{j_{\pi_K}}).$$

- For category-level tasks, compare $y_{d^{\backslash *}}$ with y_Q to update **Recall@1**; for object-level tasks, use the full ranked list for AP and mAP.



Sort the ranked vector to get the best/ closest image to our query!!

Performance Metrics

- Let $\mathcal{Q}$ be the set of **query images** (e.g. test split).
- For each query $q \in \mathcal{Q}$, retrieval returns ranked indices

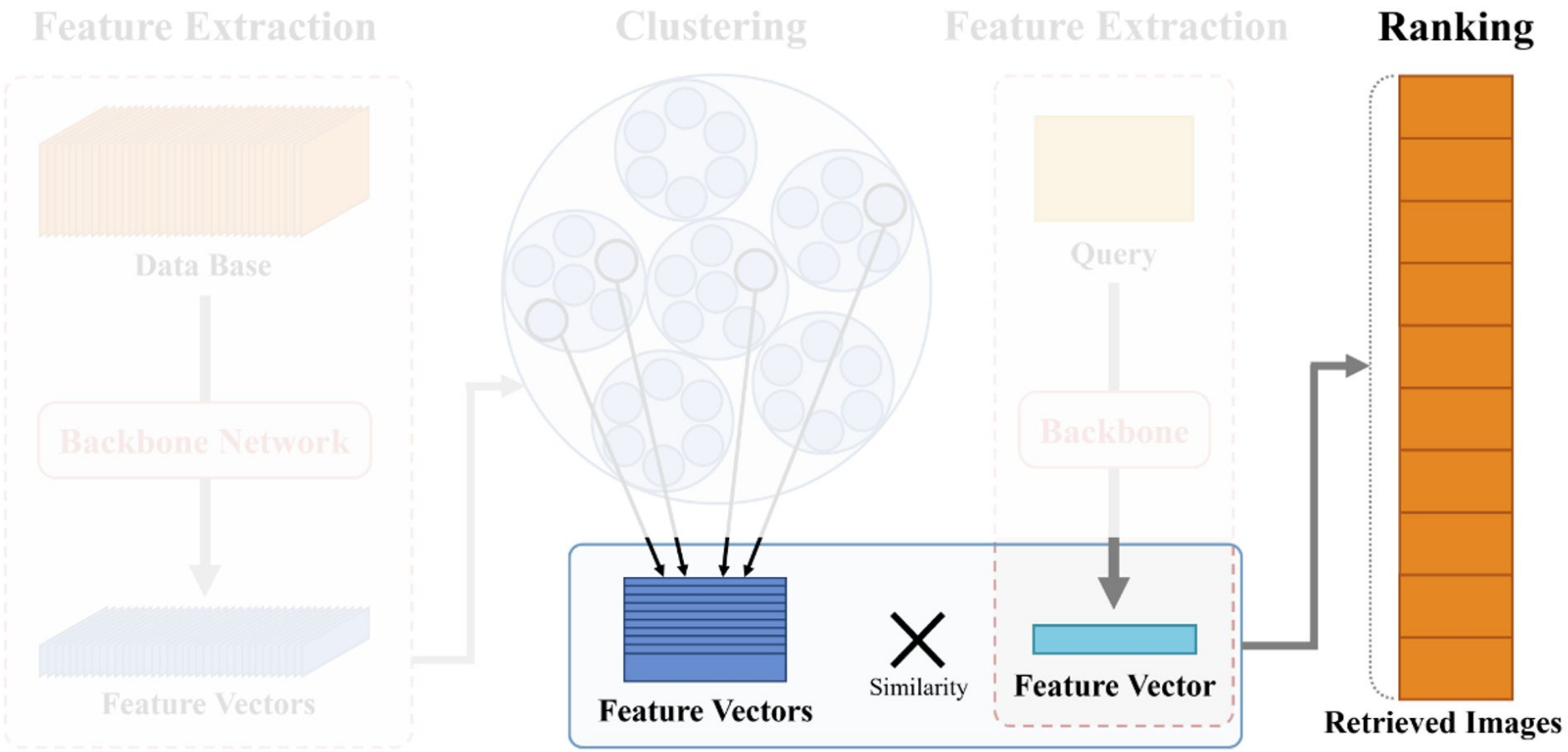
$$(r_1(q), r_2(q), \dots), \quad r_1(q) = \text{top-1 index}.$$

Recall@1 -> [category level retrieval]	mAP (mean Average Precision) -> [object level retrieval]
$$\text{Recall@1} = \frac{1}{\lvert\mathcal{Q}\rvert} \sum_{q \in \mathcal{Q}} \mathbf{1}\left[y_{r_1(q)} = y_q\right]$$	$$\text{AP}(q) = \frac{1}{R_q} \sum_{k=1}^{K} P_q(k)\, \mathbf{1}\left[\text{ item at rank } k \text{ is relevant}\right]$$
• Fraction of queries where the top retrieved image has the correct label. • Absolute aggregation happens in this method.	• R_q is number of relevant images and P_q is Precision@k. • Then mAP takes the mean over all the queries.

OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

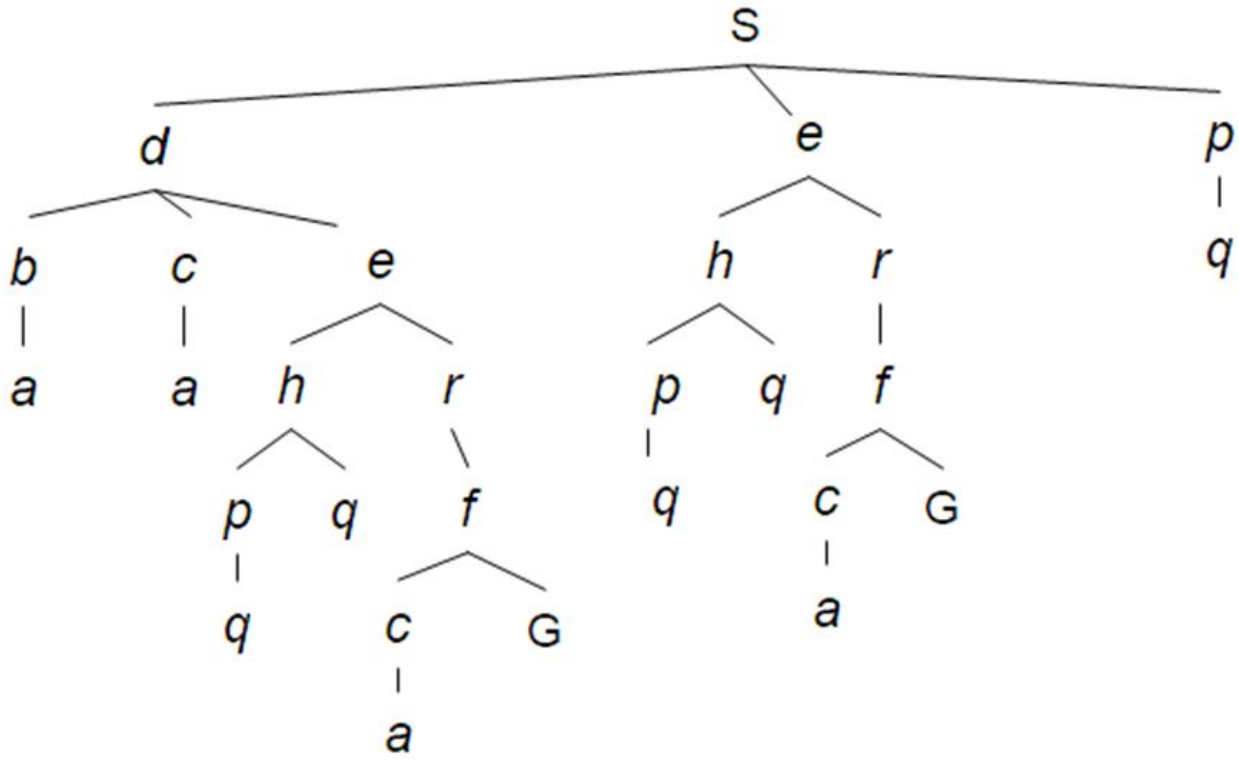
Online Step



- **Process:** Tree Traversal → Distance Calculation → Ranking

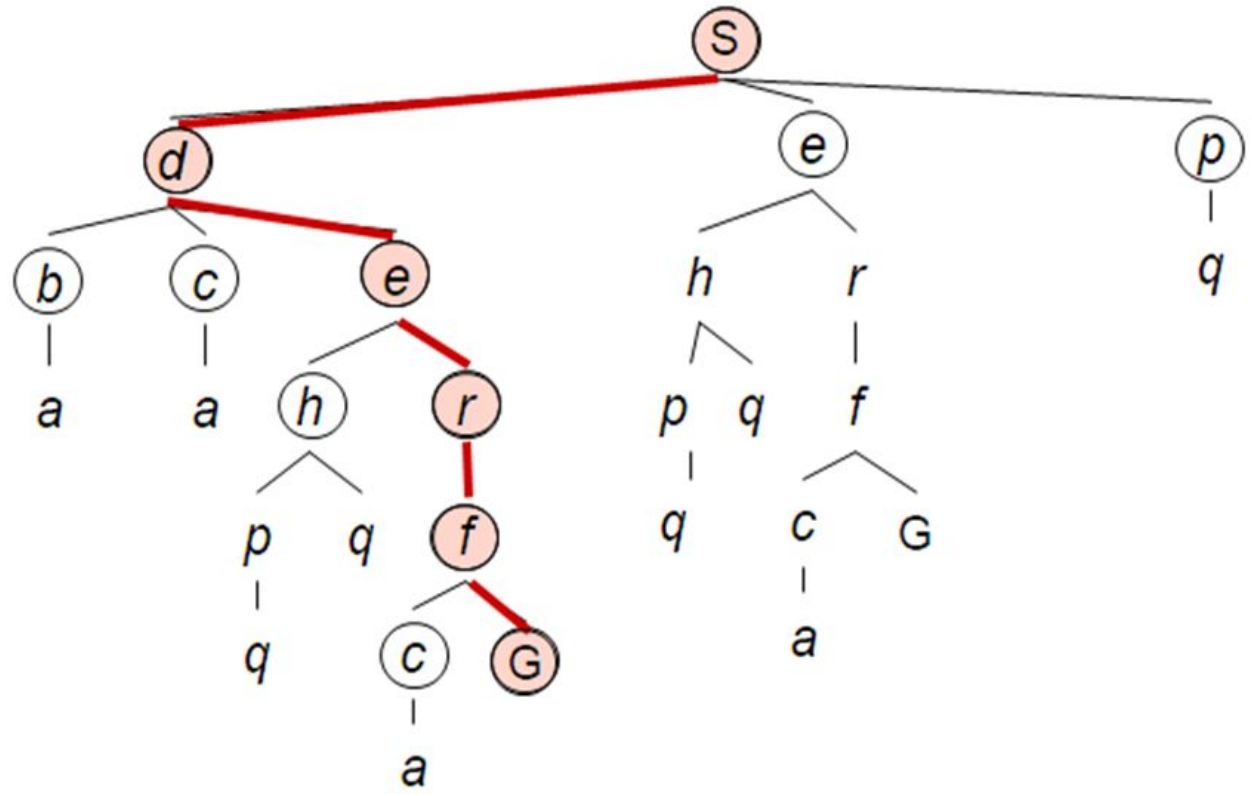
Accuracy vs. Compute Trade-off

- The Offline Step constructs a hierarchical K-means tree to index global descriptors
- Traverse the pre-built Hierarchical K-Means tree to find candidate leaf nodes



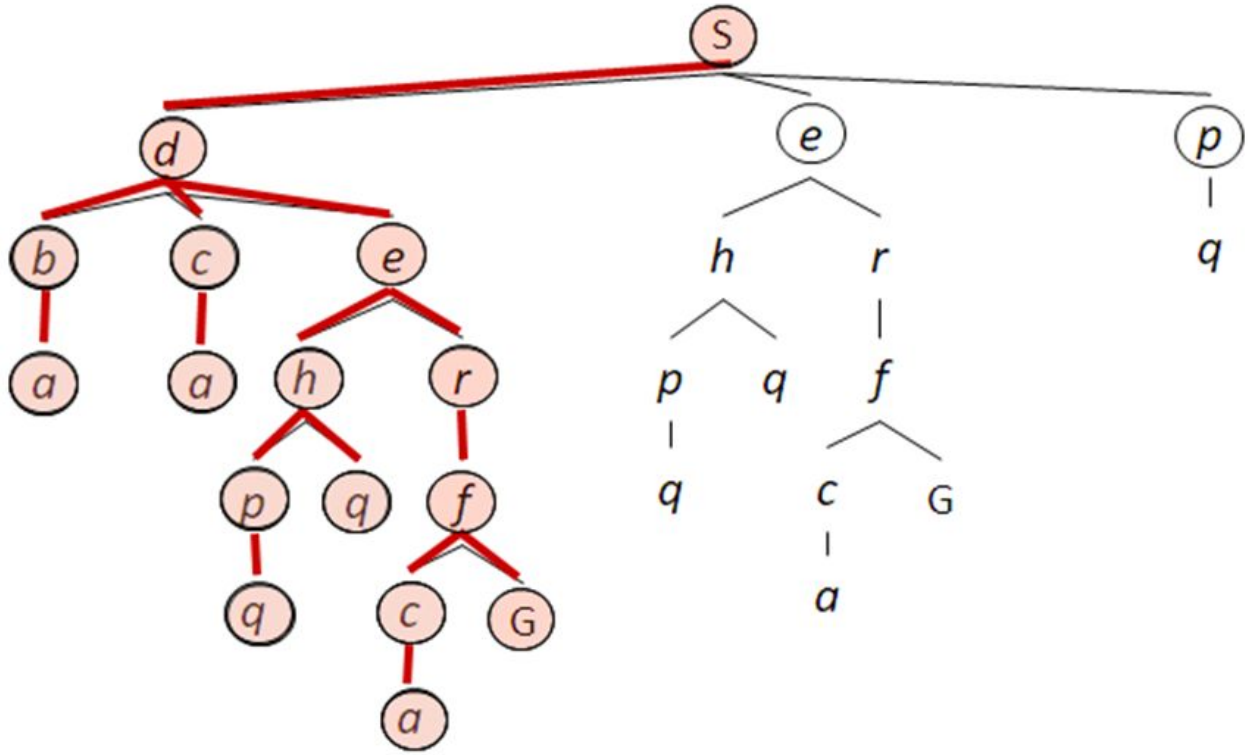
Accuracy vs. Compute Trade-off

- Selecting the closest node is computationally efficient but risky
- Neighboring clusters that are further from the query centroid but contain the target image



Accuracy vs. Compute Trade-off

- To develop an "Online Step" that balances the trade-off between **retrieval speed** (pruning the tree) and **retrieval accuracy** (exploring multiple branches)



Search Algorithms

I. Intensive Search
II. Relaxed Search
III. Auto Search

Search Algorithms

I. Intensive Search
II. Relaxed Search
III. Auto Search

Intensive Search

Prioritize maximum retrieval speed.

- **Mechanism:**
  - Select **one** node per level.
  - Centroid vector closest to the query descriptor.

$$i_{selected} = \underset{i \in \text{CurrentLevel}}{\text{argmin}} \; \Phi(\mu_i, e_q)$$

  - Children of unselected nodes are excluded.
  - Identifies a single leaf node.

Fastest execution at the cost of accuracy loss.

Search Algorithms

I. Intensive Search
II. Relaxed Search
III. Auto Search

Relaxed Search

Traverse **multiple branches** → Max. accuracy retention.

- **Mechanism:**
  - multiple nodes (N^m) at each level based on two hyperparameters:
    - s_1 (Absolute Threshold)**:** Selects nodes within distance s_1 of the query.

$$N' = \{i \mid \Phi(\mu_i, e_q) \leq s_1, i \in N^m\}$$

    - s_2 (Relative Margin)**:** Selects nodes within margin s_2 of the closest node.

$$N'' = \{i \mid \Phi(\mu_i, e_q) - \min_{j \in N^m} \Phi(\mu_j, e_q) \leq s_2, i \in N^m\}$$

Explores a broader set of candidate leaf nodes. $N^m \leftarrow N' \cup N''$

Preserves retrieval accuracy but increases computational cost.

Search Algorithms

I. Intensive Search
II. Relaxed Search
III. Auto Search

Auto Search

Balance speed and accuracy and automates parameter selection

- **Mechanism:**
  - Sorts distances at the current level (D^m)

$$D^m = \left\{\Phi(\mu_j, e_q)\right\}_{j \in N^m}, \quad D_1^m \leq D_2^m \leq \cdots \leq D_{|N^m|}^m$$

  - Identifies the **largest difference (gap)** between neighbor distances. $s_1 = \underset{\theta_j}{argmax}(\theta_{j+1} - \theta_j)$
  - Dynamically sets the threshold $\boldsymbol{s_1}$ to this gap value.
    - Eliminates the need for $\boldsymbol{s_2}$ and manual tuning.

Faster than Relaxed, more accurate than Intensive.

OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

Advantages

- **Large speedup with high accuracy**

  For particular object retrieval, mAP retention stays between about 92.5% and 102%, with an average of 97.8%.
- **Flexible trade-off**:
  - Intensive search: **maximum speed.**
  - Relaxed search: **best accuracy retention.**
  - Auto search: **balanced speed and accuracy, no extra hyperparameters.**
- Works with different backbones and tasks:
  - CLIP-based model (UNICOM) for category-level retrieval.
  - CNN-based model (R-GeM) for particular object retrieval.

  Shows that **the framework is model-agnostic and reusable**

Advantages

- **Scales well with dataset size:** The larger the database, the more retrieval time is reduced
- Offline/online separation: Tree construction is done offline, online step becomes much faster and can be plugged into existing CBIR pipelines without retraining the backbone.

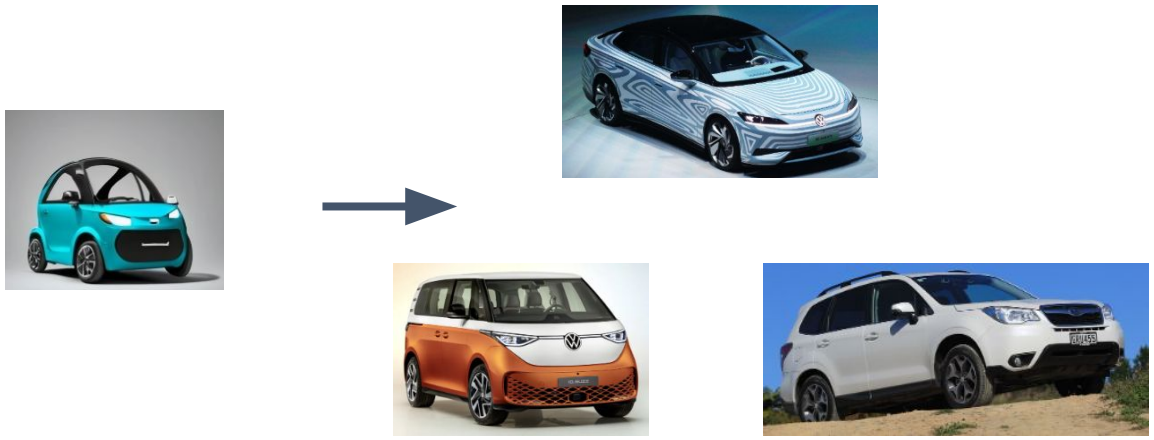
Disadvantages

- **Speed–accuracy trade-off**: Intensive search gives the highest speed but can significantly hurt accuracy on large datasets (big drop in Recall@1 on In-Shop).
- **Less benefit on small datasets or dynamic data**
- The cluster tree is built offline, when many new images arrive the tree may need to be rebuilt which is **less ideal for highly dynamic systems.**
- Implementation-level limitation: All experiments use Python, **the authors note that using a compiled language could further reduce overhead and improve speed beyond the reported numbers.**

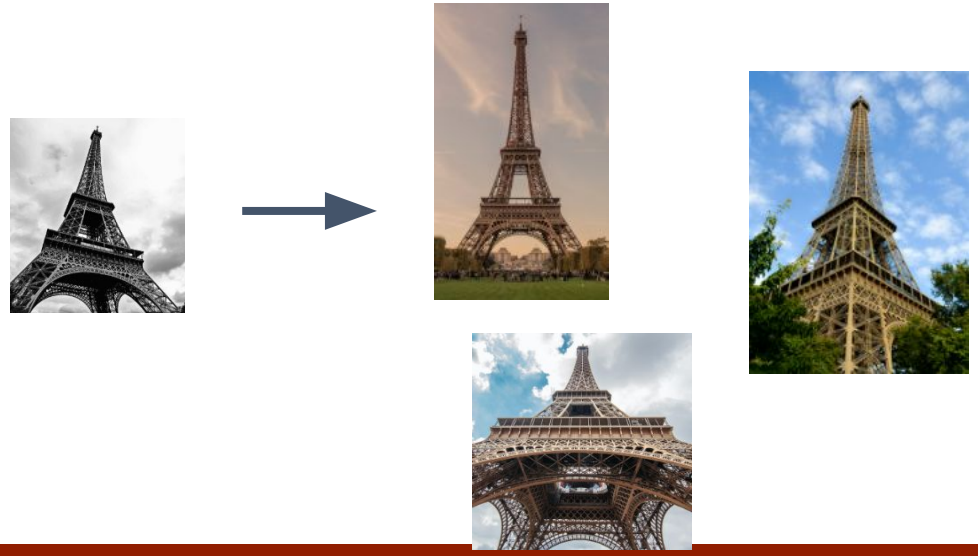
Experiment

Test Dataset	Number of Database Images	Classes	Query	
CUB [51]	5924	100	5924	Category-Level Retrieval
CARS [52]	8131	98	8131	
In-Shop [53]	12,612	3985	14,218	
$\mathcal{R}$Oxford [56]	4993	13	70	Particular Object Retrieval
$\mathcal{R}$Paris [56]	6322	12	70	
Oxford [54,55]	5063	11	55	
Paris [55]	6392	11	55	

Category-Level Retrieval



Particular Object Retrieval



OUTLINE

1. Introduction & Data Selection
2. Mechanics
3. Accuracy vs compute tradeoff
4. Advantages, Disadvantages and Experimental Results
5. Future directions

Real-World Deployment: 3 major large-scale retrieval methods

Retrieval efficiency is the key to excellent image retrieval.

Method	Core Concept	Applications
A. ANN (HNSW, IVF)	• Fast similarity search using **graph- or cluster-based structures**.	Large-scale search systems: • Internet image search (hundreds of millions) • E-commerce recommendations
B. Deep Hashing	• Train a DNN to directly output **compact binary hash codes.**	Mobile/edge devices: • On-device photo deduplication
C. Product Quantization	• Vector compression by subspace quantization.	• Cloud storage large-scale archives

ANN: Approximate Nearest Neighbor

HNSW: Hierarchical Navigable Small World IVF: Inverted File Index

ANN (Approximate Nearest Neighbors Search) Overview

- **Purpose**: Quickly find the top-K most similar points to a query in large-scale data (vectors)
- **Categories**:
  - Tree-based
  - Graph-based
  - Hashing-based

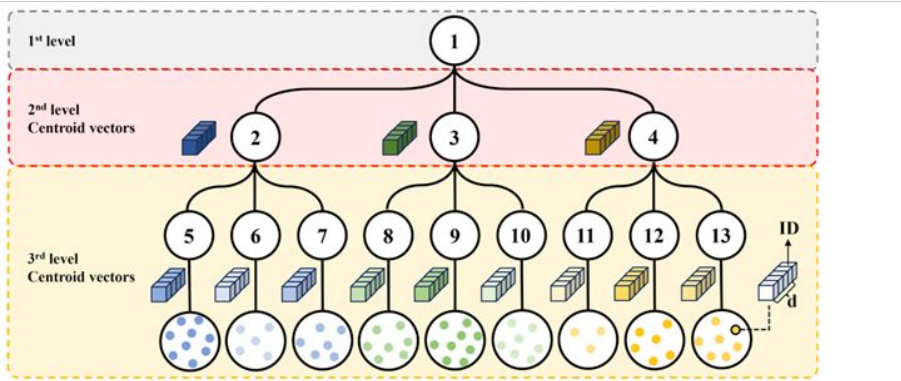
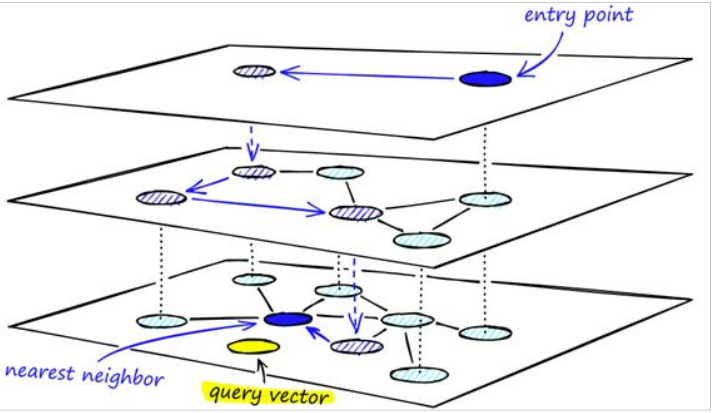
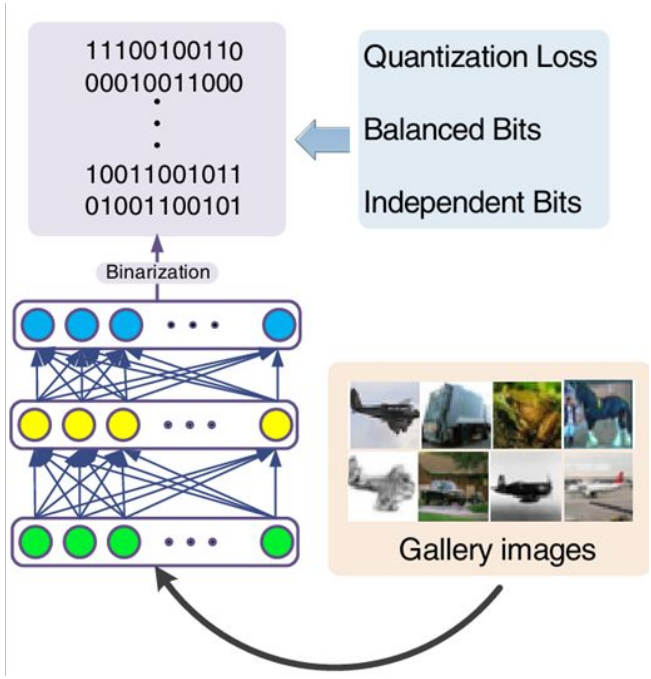


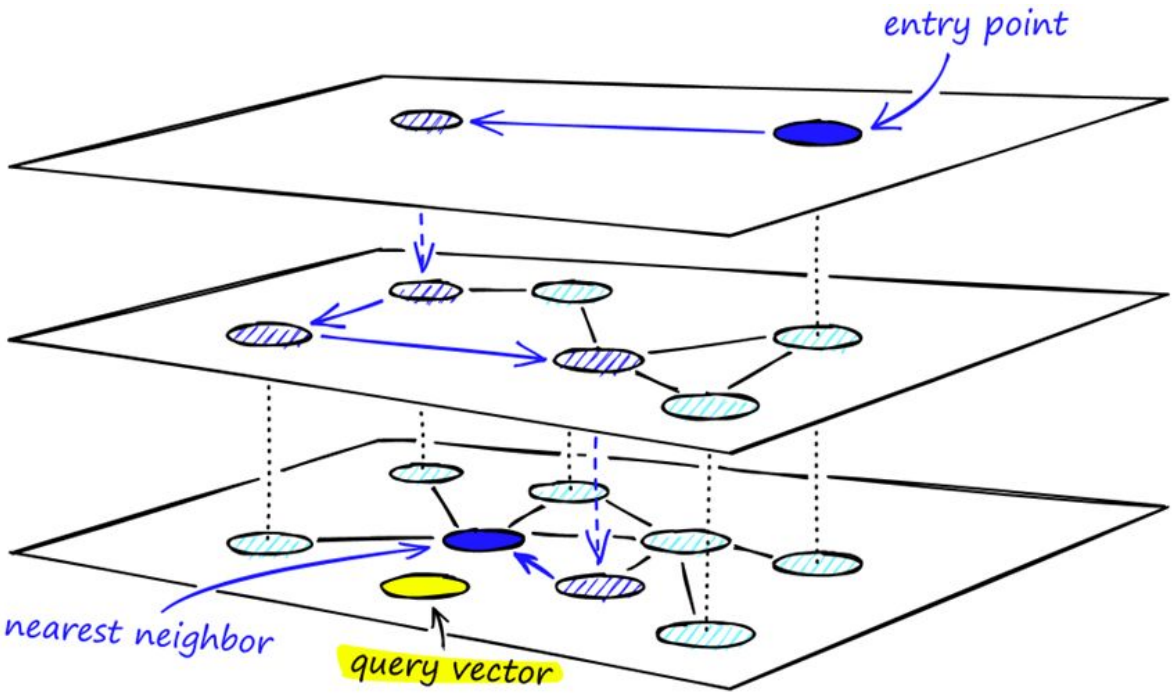
Figure 2. Structure of the tree generated in the offline step. This is an example for $k = 3, l = 3$. d is the size of a database descriptor.





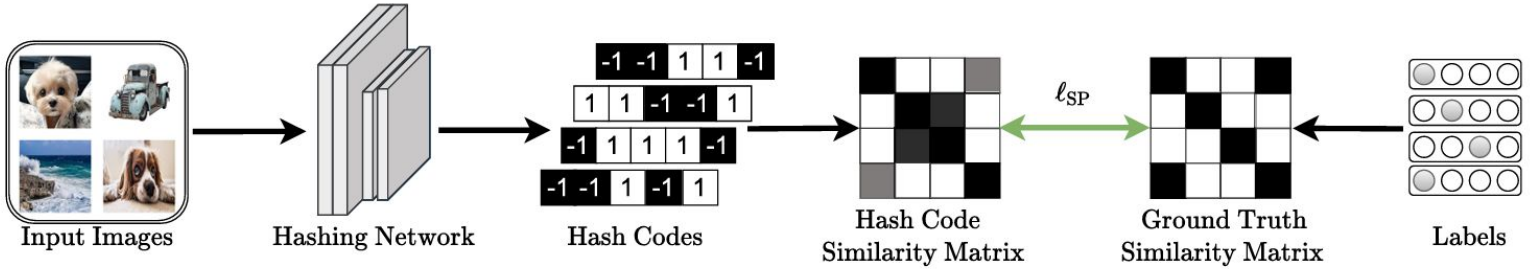
Example1: HNSW (Hierarchical Navigable Small World)

- **Industry standard:** excellent trade-off between high recall and low latency
- **Hierarchical structure:** fast coarse search + accurate local search



Example 2: Deep Hashing

- **What**: DNNs + Hashing → **convert each image into a binary hash code**
- **Why**:
  - Dramatically reduce storage requirements and enable extremely fast comparisons;
  - Suitable for **million–billion scale** databases.



$$h = Wf(x) + b \longrightarrow \mathbf{b} = \text{sign}(h)$$

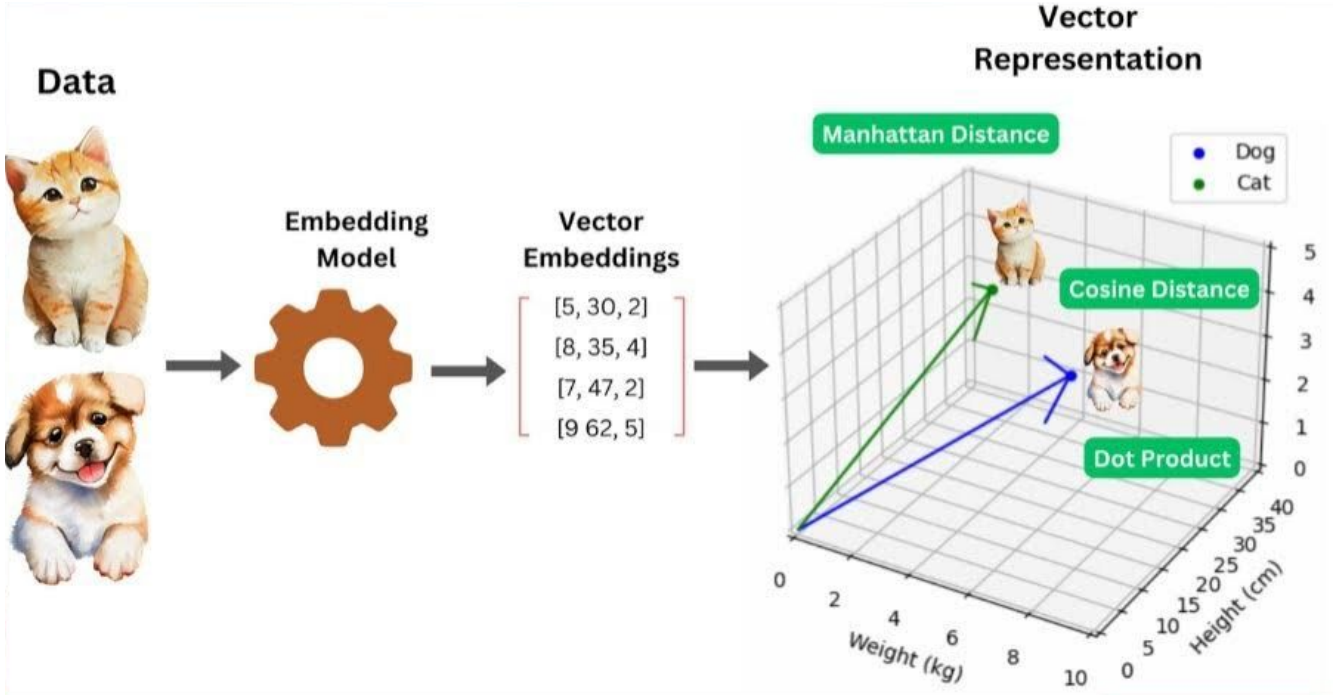
$$\|b - h\|_2^2$$

```
h = [
  [ 0.32, -1.55, 0.91, ...],   # 64 dim
  [-0.03,  0.12, -0.77, ...],
]
```

↓

```
b = [
  [ 1, -1,  1, ...],    # 64 bit binary code
  [-1,  1, -1, ...],
]
```

Content-based Image Retrieval



Q&A